

22. Find the output string generated by the Moore machine in Exercise 20 with each of these input strings.
 a) 0101 b) 111111 c) 11101110111
23. Find the output string generated by the Moore machine in Exercise 21 with each of the input strings in Exercise 22.
24. Construct a Moore machine that gives an output of 1 whenever the number of symbols in the input string read so far is divisible by 4 and an output of 0 otherwise.
25. Construct a Moore machine that determines whether an input string contains an even or odd number of 1s. The machine should give 1 as output if an even number of 1s are in the string and 0 as output if an odd number of 1s are in the string.

13.3 Finite-State Machines with No Output

13.3.1 Introduction



One of the most important applications of finite-state machines is in language recognition. This application plays a fundamental role in the design and construction of compilers for programming languages. In Section 13.2 we showed that a finite-state machine with output can be used to recognize a language, by giving an output of 1 when a string from the language has been read and a 0 otherwise. However, there are other types of finite-state machines that are specially designed for recognizing languages. Instead of producing output, these machines have final states. A string is recognized if and only if it takes the starting state to one of these final states.

13.3.2 Set of Strings

Before discussing finite-state machines with no output, we will introduce some important background material on sets of strings. The operations that will be defined here will be used extensively in our discussion of language recognition by finite-state machines.

Definition 1

Suppose that A and B are subsets of V^* , where V is a vocabulary. The *concatenation* of A and B , denoted by AB , is the set of all strings of the form xy , where x is a string in A and y is a string in B .

EXAMPLE 1 Let $A = \{0, 11\}$ and $B = \{1, 10, 110\}$. Find AB and BA .

Solution: The set AB contains every concatenation of a string in A and a string in B . Hence, $AB = \{01, 010, 0110, 111, 1110, 11110\}$. The set BA contains every concatenation of a string in B and a string in A . Hence, $BA = \{10, 111, 100, 1011, 1100, 11011\}$. ◀

Note that it is not necessarily the case that $AB = BA$ when A and B are subsets of V^* , where V is an alphabet, as Example 1 illustrates.

From the definition of the concatenation of two sets of strings, we can define A^n , for $n = 0, 1, 2, \dots$. This is done recursively by specifying that

$$\begin{aligned} A^0 &= \{\lambda\}, \\ A^{n+1} &= A^n A \quad \text{for } n = 0, 1, 2, \dots \end{aligned}$$

EXAMPLE 2 Let $A = \{1, 00\}$. Find A^n for $n = 0, 1, 2$, and 3.

Solution: We have $A^0 = \{\lambda\}$ and $A^1 = A^0 A = \{\lambda\} A = \{1, 00\}$. To find A^2 we take concatenations of pairs of elements of A . This gives $A^2 = \{11, 100, 001, 0000\}$. To find A^3 we

take concatenations of elements in A^2 and A ; this gives $A^3 = \{111, 1100, 1001, 10000, 0011, 00100, 00001, 000000\}$. ◀

Definition 2 Suppose that A is a subset of V^* . Then the *Kleene closure* of A , denoted by A^* , is the set consisting of concatenations of arbitrarily many strings from A . That is, $A^* = \bigcup_{k=0}^{\infty} A^k$.

EXAMPLE 3 What are the Kleene closures of the sets $A = \{0\}$, $B = \{0, 1\}$, and $C = \{11\}$?

Solution: The Kleene closure of A is the concatenation of the string 0 with itself an arbitrary finite number of times. Hence, $A^* = \{0^n \mid n = 0, 1, 2, \dots\}$. The Kleene closure of B is the concatenation of an arbitrary number of strings, where each string is either 0 or 1. This is the set of all strings over the alphabet $V = \{0, 1\}$. That is, $B^* = V^*$. Finally, the Kleene closure of C is the concatenation of the string 11 with itself an arbitrary number of times. Hence, C^* is the set of strings consisting of an even number of 1s. That is, $C^* = \{1^{2n} \mid n = 0, 1, 2, \dots\}$. ◀

13.3.3 Finite-State Automata

We will now give a definition of a finite-state machine with no output. Such machines are also called **finite-state automata**, and that is the terminology we will use for them here. (*Note:* The singular of *automata* is *automaton*.) These machines differ from the finite-state machines studied in Section 13.2 in that they do not produce output, but they do have a set of final states. As we will see, they recognize strings that take the starting state to a final state.

Definition 3 A *finite-state automaton* $M = (S, I, f, s_0, F)$ consists of a finite set S of *states*, a finite *input alphabet* I , a *transition function* f that assigns a next state to every pair of state and input (so that $f : S \times I \rightarrow S$), an *initial* or *start state* s_0 , and a subset F of S consisting of *final* (or *accepting states*).

We can represent finite-state automata using either state tables or state diagrams. Final states are indicated in state diagrams by using double circles.

EXAMPLE 4 Construct the state diagram for the finite-state automaton $M = (S, I, f, s_0, F)$, where $S = \{s_0, s_1, s_2, s_3\}$, $I = \{0, 1\}$, $F = \{s_0, s_3\}$, and the transition function f is given in Table 1.

Solution: The state diagram is shown in Figure 1. Note that because both the inputs 0 and 1 take s_2 to s_0 , we write 0,1 over the edge from s_2 to s_0 . ◀

Links

STEPHEN COLE KLEENE (1909–1994) Stephen Kleene was born in Hartford, Connecticut. His mother, Alice Lena Cole, was a poet, and his father, Gustav Adolph Kleene, was an economics professor. Kleene attended Amherst College and received his Ph.D. from Princeton in 1934, where he studied under the famous logician Alonzo Church. Kleene joined the faculty of the University of Wisconsin in 1935, where he remained except for several leaves, including stays at the Institute for Advanced Study in Princeton. During World War II he was a navigation instructor at the Naval Reserve's Midshipmen's School and later served as the director of the Naval Research Laboratory. Kleene made significant contributions to the theory of recursive functions, investigating questions of computability and decidability, and proved one of the central results of automata theory. He served as the Acting Director of the Mathematics Research Center and as Dean of the College of Letters and Sciences at the University of Wisconsin. Kleene was a student of natural history. He discovered a previously undescribed variety of butterfly that is named after him. He was an avid hiker and climber. Kleene was also noted as a talented teller of anecdotes, using a powerful voice that could be heard several offices away.

TABLE 1		
State	f	
	Input	
	0	1
s_0	s_0	s_1
s_1	s_0	s_2
s_2	s_0	s_0
s_3	s_2	s_1

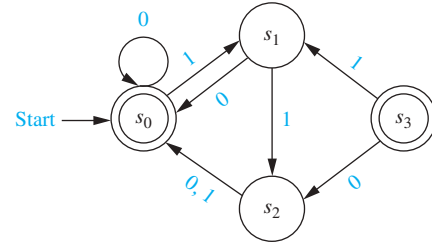


FIGURE 1 The state diagram for a finite-state automaton.

EXTENDING THE TRANSITION FUNCTION The transition function f of a finite-state machine $M = (S, I, f, s_0, F)$ can be extended so that it is defined for all pairs of states and strings; that is, f can be extended to a function $f : S \times I^* \rightarrow S$. Let $x = x_1x_2 \dots x_k$ be a string in I^* . Then $f(s_1, x)$ is the state obtained by using each successive symbol of x , from left to right, as input, starting with state s_1 . From s_1 we go on to state $s_2 = f(s_1, x_1)$, then to state $s_3 = f(s_2, x_2)$, and so on, with $f(s_1, x) = f(s_k, x_k)$. Formally, we can define this extended transition function f recursively for the deterministic finite-state machine $M = (S, I, f, s_0, F)$ by

- (i) $f(s, \lambda) = s$ for every state $s \in S$; and
- (ii) $f(s, xa) = f(f(s, x), a)$ for all $s \in S$, $x \in I^*$, and $a \in I$.

We can use structural induction and this recursive definition to prove properties of this extended transition function. For example, in Exercise 15 we ask you to prove that

$$f(s, xy) = f(f(s, x), y)$$

for every state $s \in S$ and strings $x \in I^*$ and $y \in I^*$.

13.3.4 Language Recognition by Finite-State Machines

Next, we define some terms that are used when studying the recognition by finite-state automata of certain sets of strings.

Definition 4

A string x is said to be *recognized* or *accepted* by the machine $M = (S, I, f, s_0, F)$ if it takes the initial state s_0 to a final state, that is, $f(s_0, x)$ is a state in F . The *language recognized* or *accepted* by the machine M , denoted by $L(M)$, is the set of all strings that are recognized by M . Two finite-state automata are called *equivalent* if they recognize the same language.

In Example 5 we will find the languages recognized by several finite-state automata.

EXAMPLE 5 Determine the languages recognized by the finite-state automata M_1 , M_2 , and M_3 in Figure 2.

Solution: The only final state of M_1 is s_0 . The strings that take s_0 to itself are those consisting of zero or more consecutive 1s. Hence, $L(M_1) = \{1^n \mid n = 0, 1, 2, \dots\}$.

The only final state of M_2 is s_2 . The only strings that take s_0 to s_2 are 1 and 01. Hence, $L(M_2) = \{1, 01\}$.

The final states of M_3 are s_0 and s_3 . The only strings that take s_0 to itself are λ , 0, 00, 000, $\dots$, that is, any string of zero or more consecutive 0s. The only strings that take s_0 to s_3 are a string of zero or more consecutive 0s, followed by 10, followed by any string. Hence, $L(M_3) = \{0^n, 0^n10x \mid n = 0, 1, 2, \dots, \text{ and } x \text{ is any string}\}$. ◀

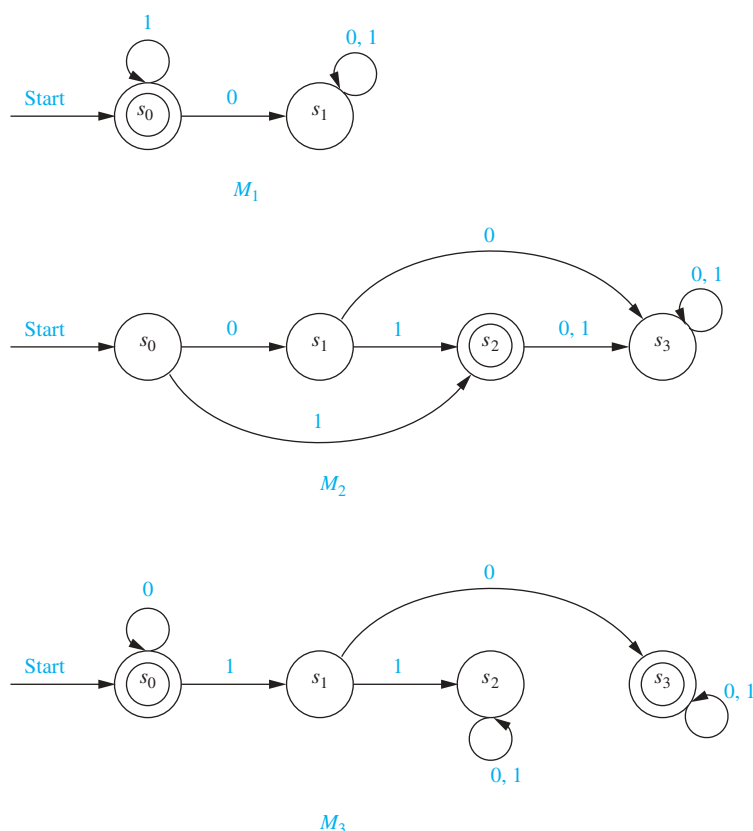


FIGURE 2 Some finite-state automata.

DESIGNING FINITE-STATE AUTOMATA We can often construct a finite-state automaton that recognizes a given set of strings by carefully adding states and transitions and determining which of these states should be final states. When appropriate we include states that can keep track of some of the properties of the input string, providing the finite-state automaton with limited memory. Examples 6 and 7 illustrate some of the techniques that can be used to construct finite-state automata that recognize particular types of sets of strings.

EXAMPLE 6 Construct deterministic finite-state automata that recognize each of these languages.

Extra Examples ➤

- the set of bit strings that begin with two 0s
- the set of bit strings that contain two consecutive 0s
- the set of bit strings that do not contain two consecutive 0s
- the set of bit strings that end with two 0s
- the set of bit strings that contain at least two 0s

Solution: (a) Our goal is to construct a deterministic finite-state automaton that recognizes the set of bit strings that begin with two 0s. Besides the start state s_0 , we include a nonfinal state s_1 ; we move to s_1 from s_0 if the first bit is a 0. Next, we add a final state s_2 , which we move to from s_1 if the second bit is a 0. When we have reached s_2 we know that the first two input bits are both 0s, so we stay in the state s_2 no matter what the succeeding bits (if any) are. We move to a nonfinal state s_3 from s_0 if the first bit is a 1 and from s_1 if the second bit is a 1. The reader should verify that the finite-state automaton in Figure 3(a) recognizes the set of bit strings that begin with two 0s.

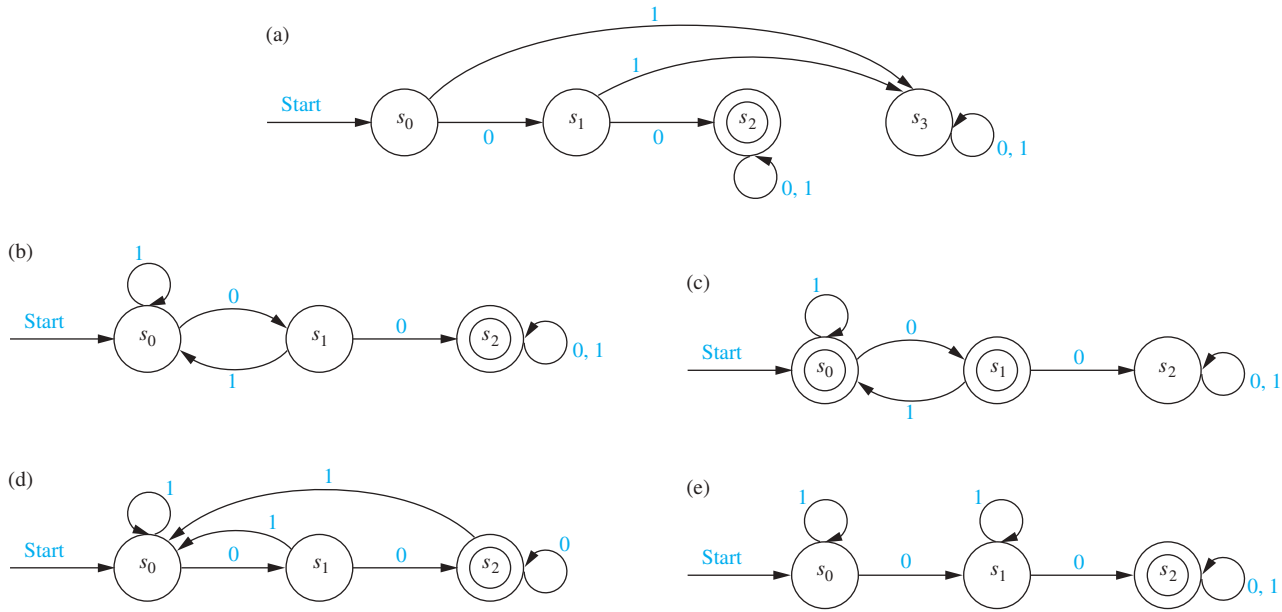


FIGURE 3 Deterministic finite-state automata recognizing the languages in Example 6.

(b) Our goal is to construct a deterministic finite-state automaton that recognizes the set of bit strings that contain two consecutive 0s. Besides the start state s_0 , we include a nonfinal state s_1 , which tells us that the last input bit seen is a 0, but either the bit before it was a 1, or this bit was the initial bit of the string. We include a final state s_2 that we move to from s_1 when the next input bit after a 0 is also a 0. If a 1 follows a 0 in the string (before we encounter two consecutive 0s), we return to s_0 and begin looking for consecutive 0s all over again. The reader should verify that the finite-state automaton in Figure 3(b) recognizes the set of bit strings that contain two consecutive 0s.

(c) Our goal is to construct a deterministic finite-state automaton that recognizes the set of bit strings that do not contain two consecutive 0s. Besides the start state s_0 , which should be a final state, we include a final state s_1 , which we move to from s_0 when 0 is the first input bit. When an input bit is a 1, we return to, or stay in, state s_0 . We add a state s_2 , which we move to from s_1 when the input bit is a 0. Reaching s_2 tells us that we have seen two consecutive 0s as input bits. We stay in state s_2 once we have reached it; this state is not final. The reader should verify that the finite-state automaton in Figure 3(c) recognizes the set of bit strings that do not contain two consecutive 0s. [The astute reader will notice the relationship between the finite-state automaton constructed here and the one constructed in part (b). See Exercise 39.]

(d) Our goal is to construct a deterministic finite-state automaton that recognizes the set of bit strings that end with two 0s. Besides the start state s_0 , we include a nonfinal state s_1 , which we move to if the first bit is 0. We include a final state s_2 , which we move to from s_1 if the next input bit after a 0 is also a 0. If an input of 0 follows a previous 0, we stay in state s_2 because the last two input bits are still 0s. Once we are in state s_2 , an input bit of 1 sends us back to s_0 , and we begin looking for consecutive 0s all over again. We also return to s_0 if the next input is a 1 when we are in state s_1 . The reader should verify that the finite-state automaton in Figure 3(d) recognizes the set of bit strings that end with two 0s.

(e) Our goal is to construct a deterministic finite-state automaton that recognizes the set of bit strings that contain two 0s. Besides the start state, we include a state s_1 , which is not final; we stay in s_0 until an input bit is a 0 and we move to s_1 when we encounter the first 0 bit in the input. We add a final state s_2 , which we move to from s_1 once we encounter a second 0 bit. Whenever we encounter a 1 as input, we stay in the current state. Once we have reached s_2 , we remain

there. Here, s_1 and s_2 are used to tell us that we have already seen one or two 0s in the input string so far, respectively. The reader should verify that the finite-state automaton in Figure 3(e) recognizes the set of bit strings that contain two 0s. ◀

EXAMPLE 7 Construct a deterministic finite-state automaton that recognizes the set of bit strings that contain an odd number of 1s and that end with at least two consecutive 0s.

Solution: We can build a deterministic finite-state automaton that recognizes the specified set by including states that keep track of both the parity of the number of 1 bits and whether we have seen no, one, or at least two 0s at the end of the input string.

The start state s_0 can be used to tell us that the input read so far contains an even number of 1s and ends with no 0s (that is, is empty or ends with a 1). Besides the start state, we include five more states. We move to states s_1 , s_2 , s_3 , s_4 , and s_5 , respectively, when the input string read so far contains an even number of 1s and ends with one 0; when it contains an even number of 1s and ends with at least two 0s; when it contains an odd number of 1s and ends with no 0s; when it contains an odd number of 1s and ends with one 0; and when it contains an odd number of 1s and ends with two 0s. The state s_5 is a final state.

The reader should verify that the finite-state automaton in Figure 4 recognizes the set of bit strings that contain an odd number of 1s and end with at least two consecutive 0s. ◀

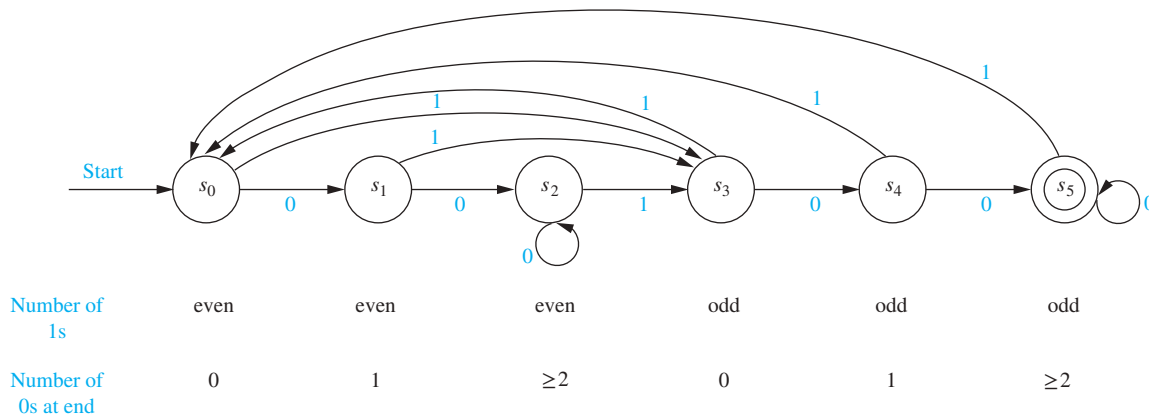


FIGURE 4 A deterministic finite-state automaton recognizing the set of bit strings containing an odd number of 1s and ending with at least two 0s.

EQUIVALENT FINITE-STATE AUTOMATA In Definition 4 we specified that two finite-state automata are equivalent if they recognize the same language. Example 8 provides an example of two equivalent deterministic finite-state machines.

EXAMPLE 8 Show that the two finite-state automata M_0 and M_1 shown in Figure 5 are equivalent.

Solution: For a string x to be recognized by M_0 , x must take us from s_0 to the final state s_1 or the final state s_4 . The only string that takes us from s_0 to s_1 is the string 1. The strings that take us from s_0 to s_4 are those strings that begin with a 0, which takes us from s_0 to s_2 , followed by zero or more additional 0s, which keep the machine in state s_2 , followed by a 1, which takes us from state s_2 to the final state s_4 . All other strings take us from s_0 to a state that is not final. (We leave it to the reader to fill in the details.) We conclude that $L(M_0)$ is the set of strings of zero or more 0 bits followed by a final 1.

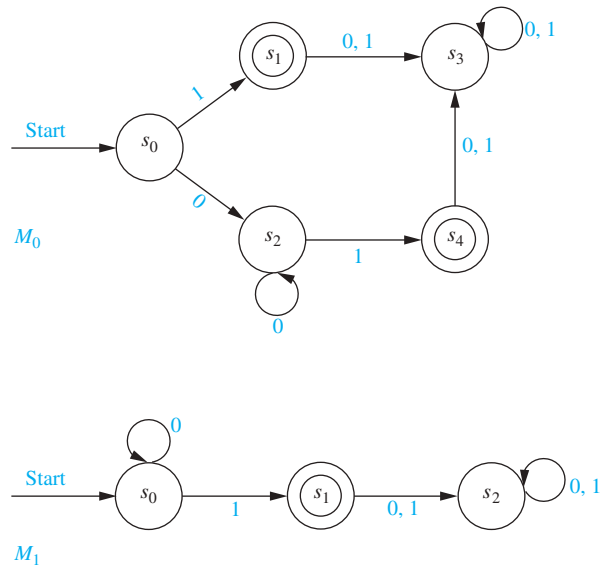


FIGURE 5 M_0 and M_1 are equivalent finite-state automata.

For a string x to be recognized by M_1 , x must take us from s_0 to the final state s_1 . So, for x to be recognized, it must begin with some number of 0s, which leave us in state s_0 , followed by a 1, which takes us to the final state s_1 . A string of all zeros is not recognized because it leaves us in state s_0 , which is not final. All strings that contain a 0 after 1 are not recognized because they take us to state s_2 , which is not final. It follows that $L(M_1)$ is the same as $L(M_0)$. We conclude that M_0 and M_1 are equivalent.

Note that the finite-state machine M_1 only has three states. No finite state machine with fewer than three states can be used to recognize the set of all strings of zero or more 0 bits followed by a 1 (see Exercise 37). ◀

As Example 8 shows, a finite-state automaton may have more states than one equivalent to it. In fact, algorithms used to construct finite-state automata to recognize certain languages may have many more states than necessary. Using unnecessarily large finite-state machines to recognize languages can make both hardware and software applications inefficient and costly. This problem arises when finite-state automata are used in compilers, which translate computer programs to a language a computer can understand (object code).

Exercises 58–61 develop a procedure that constructs a finite-state automaton with the fewest states possible among all finite-state automata equivalent to a given finite-state automaton. This procedure is known as **machine minimization**. The minimization procedure described in these exercises reduces the number of states by replacing states with equivalence classes of states with respect to an equivalence relation in which two states are equivalent if every input string either sends both states to a final state or sends both to a state that is not final. Before the minimization procedure begins, all states that cannot be reached from the start state using any input string are first removed; removing these does not change the language recognized.

13.3.5 Nondeterministic Finite-State Automata

The finite-state automata discussed so far are **deterministic**, because for each pair of state and input value there is a unique next state given by the transition function. There is another important type of finite-state automaton in which there may be several possible next states for

each pair of input value and state. Such machines are called **nondeterministic**. Nondeterministic finite-state automata are important in determining which languages can be recognized by a finite-state automaton.

Definition 5

Links 

A *nondeterministic finite-state automaton* $M = (S, I, f, s_0, F)$ consists of a set S of states, an input alphabet I , a transition function f that assigns a set of states to each pair of state and input (so that $f : S \times I \rightarrow P(S)$), a starting state s_0 , and a subset F of S consisting of the final states.

We can represent nondeterministic finite-state automata using state tables or state diagrams. When we use a state table, for each pair of state and input value we give a list of possible next states. In the state diagram, we include an edge from each state to all possible next states, labeling edges with the input or inputs that lead to this transition.

EXAMPLE 9 Find the state diagram for the nondeterministic finite-state automaton with the state table shown in Table 2. The final states are s_2 and s_3 .

Solution: The state diagram for this automaton is shown in Figure 6. 

Links 



©Bettmann/Getty Images

GRACE BREWSTER MURRAY HOPPER (1906–1992) Grace Hopper, born in New York City, displayed an intense curiosity as a child with how things worked. At the age of seven, she disassembled alarm clocks to discover their mechanisms. She inherited her love of mathematics from her mother, who received special permission to study geometry (but not algebra and trigonometry) at a time when women were actively discouraged from such study. Hopper was inspired by her father, a successful insurance broker, who had lost his legs from circulatory problems. He told his children they could do anything if they put their minds to it. He inspired Hopper to pursue higher education and not conform to the usual roles for females. Her parents made sure that she had an excellent education; she attended private schools for girls in New York. Hopper entered Vassar College in 1924, where she majored in mathematics and physics; she graduated in 1928. She received a masters degree in mathematics from Yale University in 1930. In 1930 she also married an English instructor at the New York School of Commerce; she later divorced and did not have children. Hopper was a mathematics professor at Vassar from 1931 until 1943, earning a Ph.D. from Yale in 1934.

After the attack on Pearl Harbor, Hopper, coming from a family with strong military traditions, decided to leave her academic position and join the Navy WAVES. To enlist, she needed special permission to leave her strategic position as a mathematics professor, as well as a waiver for weighing too little. In December 1943, she was sworn into the Navy Reserve and trained at the Midshipman's School for Women. Hopper was assigned to work at the Naval Ordnance Laboratory at Harvard University. She wrote programs for the world's first large-scale automatically sequenced digital computer, which was used to help aim Navy artillery in varying weather. Hopper has been credited with coining the term "bug" to refer to a hardware glitch, but it was used at Harvard prior to her arrival there. However, it is true that Hopper and her programming team found a moth in one of the relays in the computer hardware that shut the system down. This famous moth was pasted into a lab book. In the 1950s Hopper coined the term "debug" for the process of removing programming errors.

In 1946, when the Navy told her that she was too old for active service, Hopper chose to remain at Harvard as a civilian research fellow. In 1949 she left Harvard to join the Eckert–Mauchly Computer Corporation, where she helped develop UNIVAC, the first commercial computer. Hopper remained with this company when it was taken over by Remington Rand and when Remington Rand merged with the Sperry Corporation. She was a visionary for the potential power of computers; she understood that computers would become widely used if tools that were both programmer-friendly and application-friendly could be developed. In particular, she believed that computer programs could be written in English, rather than using machine instructions. To help achieve this goal, she developed the first compiler. She published the first research paper on compilers in 1952. Hopper is also known as the mother of the computer language COBOL; members of Hopper's staff helped to frame the basic language design for COBOL using their earlier work as a basis.

In 1966, Hopper retired from the Navy Reserve. However, only seven months later, the Navy recalled her from retirement to help standardize high-level naval computer languages. In 1983 she was promoted to the rank of Commodore by special Presidential appointment, and in 1985 she was elevated to the rank of Rear Admiral. Her retirement from the Navy, at the age of 80, was held on the *USS Constitution*.

TABLE 2		
State	f	
	Input	
	0	1
s_0	s_0, s_1	s_3
s_1	s_0	s_1, s_3
s_2		s_0, s_2
s_3	s_0, s_1, s_2	s_1

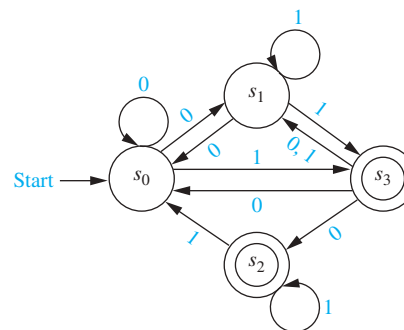


FIGURE 6 The nondeterministic finite-state automaton with state table given in Table 2.

EXAMPLE 10 Find the state table for the nondeterministic finite-state automaton with the state diagram shown in Figure 7.

Solution: The state table is given as Table 3. ◀

What does it mean for a nondeterministic finite-state automaton to recognize a string $x = x_1x_2 \dots x_k$? The first input symbol x_1 takes the starting state s_0 to a set S_1 of states. The next input symbol x_2 takes each of the states in S_1 to a set of states. Let S_2 be the union of these sets. We continue this process, including, at each stage, all states obtained using a state obtained at the previous stage and the current input symbol. We **recognize**, or **accept**, the string x if there is a final state in the set of all states that can be obtained from s_0 using x . The **language recognized** by a nondeterministic finite-state automaton is the set of all strings recognized by this automaton.

EXAMPLE 11 Find the language recognized by the nondeterministic finite-state automaton shown in Figure 7.

Solution: Because s_0 is a final state, and there is a transition from s_0 to itself when 0 is the input, the machine recognizes all strings consisting of zero or more consecutive 0s. Furthermore, because s_4 is a final state, any string that has s_4 in the set of states that can be reached from s_0 with this input string is recognized. The only such strings are strings consisting of zero or more consecutive 0s followed by 01 or 11. Because s_0 and s_4 are the only final states, the language recognized by the machine is $\{0^n, 0^n01, 0^n11 \mid n \geq 0\}$. ◀

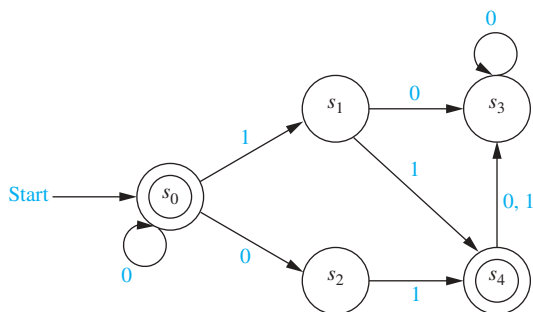


FIGURE 7 A nondeterministic finite-state automaton.

TABLE 3		
State	f	
	Input	
	0	1
s_0	s_0, s_2	s_1
s_1	s_3	s_4
s_2		s_4
s_3	s_3	
s_4	s_3	s_3

One important fact is that a language recognized by a nondeterministic finite-state automaton is also recognized by a deterministic finite-state automaton. We will take advantage of this fact in Section 13.4 when we will determine which languages are recognized by finite-state automata.

THEOREM 1

If the language L is recognized by a nondeterministic finite-state automaton M_0 , then L is also recognized by a deterministic finite-state automaton M_1 .



Proof: We will describe how to construct the deterministic finite-state automaton M_1 that recognizes L from M_0 , the nondeterministic finite-state automaton that recognizes this language. Each state in M_1 will be made up of a set of states in M_0 . The start symbol of M_1 is $\{s_0\}$, which is the set containing the start state of M_0 . The input set of M_1 is the same as the input set of M_0 .

Given a state $\{s_{i_1}, s_{i_2}, \dots, s_{i_k}\}$ of M_1 , the input symbol x takes this state to the union of the sets of next states for the elements of this set, that is, the union of the sets $f(s_{i_1}, x)$, $f(s_{i_2}, x)$, $\dots$, $f(s_{i_k}, x)$. The states of M_1 are all the subsets of S , the set of states of M_0 , that are obtained in this way starting with s_0 . (There are as many as 2^n states in the deterministic machine, where n is the number of states in the nondeterministic machine, because all subsets may occur as states, including the empty set, although usually far fewer states occur.) The final states of M_1 are those sets that contain a final state of M_0 .

Suppose that an input string is recognized by M_0 . Then one of the states that can be reached from s_0 using this input string is a final state (the reader should provide an inductive proof of this). This means that in M_1 , this input string leads from $\{s_0\}$ to a set of states of M_0 that contains a final state. This subset is a final state of M_1 , so this string is also recognized by M_1 . Also, an input string not recognized by M_0 does not lead to any final states in M_0 . (The reader should provide the details that prove this statement.) Consequently, this input string does not lead from $\{s_0\}$ to a final state in M_1 . ◀

EXAMPLE 12 Find a deterministic finite-state automaton that recognizes the same language as the nondeterministic finite-state automaton in Example 10.

Solution: The deterministic automaton shown in Figure 8 is constructed from the nondeterministic automaton in Example 10. The states of this deterministic automaton are subsets of the set of all states of the nondeterministic machine. The next state of a subset under an input symbol is the subset containing the next states in the nondeterministic machine of all elements in this subset. For instance, on input of 0, $\{s_0\}$ goes to $\{s_0, s_2\}$, because s_0 has transitions to itself and to s_2 in the nondeterministic machine; the set $\{s_0, s_2\}$ goes to $\{s_1, s_4\}$ on input of 1, because

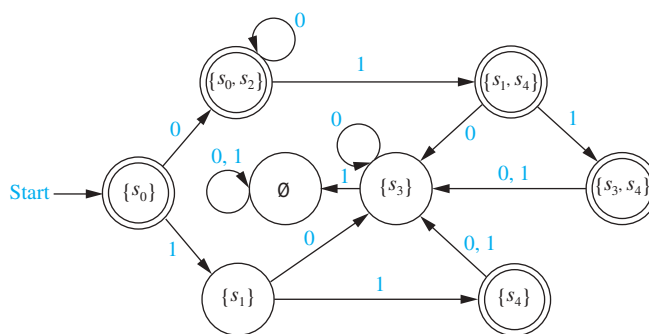
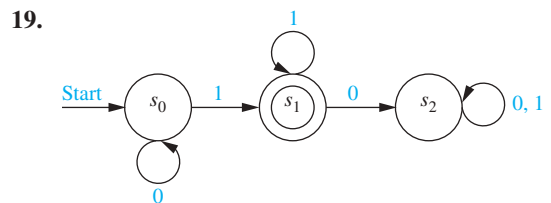
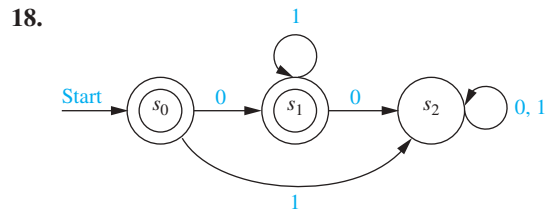
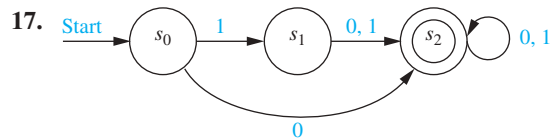
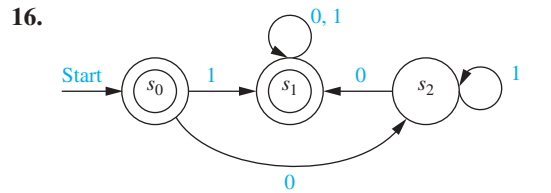


FIGURE 8 A deterministic automaton equivalent to the nondeterministic automaton in Example 10.

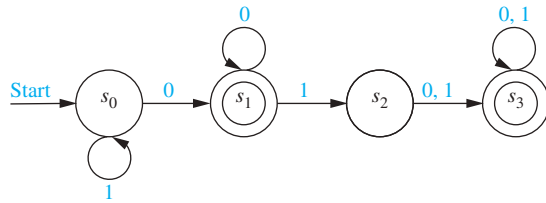
s_0 goes just to s_1 and s_2 goes just to s_4 on input of 1 in the nondeterministic machine; and the set $\{s_1, s_4\}$ goes to $\{s_3\}$ on input of 0, because s_1 and s_4 both go to just s_3 on input of 0 in the deterministic machine. All subsets that are obtained in this way are included in the deterministic finite-state machine. Note that the empty set is one of the states of this machine, because it is the subset containing all the next states of $\{s_3\}$ on input of 1. The start state is $\{s_0\}$, and the set of final states are all those that include s_0 or s_4 . ◀

Exercises

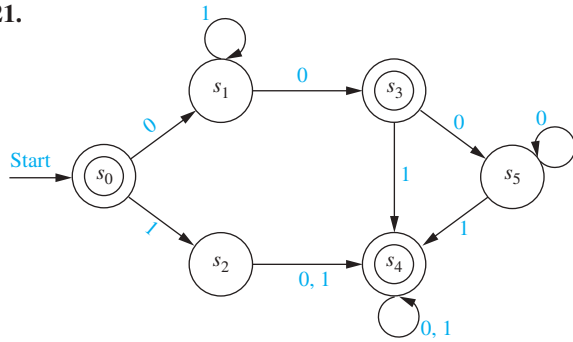
- Let $A = \{0, 11\}$ and $B = \{00, 01\}$. Find each of these sets.
a) AB b) BA c) A^2 d) B^3
 - Show that if A is a set of strings, then $A\emptyset = \emptyset A = \emptyset$.
 - Find all pairs of sets of strings A and B for which $AB = \{10, 111, 1010, 1000, 10111, 101000\}$.
 - Show that these equalities hold.
a) $\{\lambda\}^* = \{\lambda\}$
b) $(A^*)^* = A^*$ for every set of strings A
 - Describe the elements of the set A^* for these values of A .
a) $\{10\}$ b) $\{111\}$ c) $\{0, 01\}$ d) $\{1, 101\}$
 - Let V be an alphabet, and let A and B be subsets of V^* . Show that $|AB| \leq |A||B|$.
 - Let V be an alphabet, and let A and B be subsets of V^* with $A \subseteq B$. Show that $A^* \subseteq B^*$.
 - Suppose that A is a subset of V^* , where V is an alphabet. Prove or disprove each of these statements.
a) $A \subseteq A^2$ b) if $A = A^2$, then $\lambda \in A$
c) $A\{\lambda\} = A$ d) $(A^*)^* = A^*$
e) $A^*A = A^*$ f) $|A^n| = |A|^n$
 - Determine whether the string 11101 is in each of these sets.
a) $\{0, 1\}^*$ b) $\{1\}^*\{0\}^*\{1\}^*$
c) $\{11\}\{0\}^*\{01\}$ d) $\{11\}^*\{01\}^*$
e) $\{111\}^*\{0\}^*\{1\}$ f) $\{11, 0\}\{00, 101\}$
 - Determine whether the string 01001 is in each of these sets.
a) $\{0, 1\}^*$ b) $\{0\}^*\{10\}\{1\}^*$
c) $\{010\}^*\{0\}^*\{1\}$ d) $\{010, 011\}\{00, 01\}$
e) $\{00\}\{0\}^*\{01\}$ f) $\{01\}^*\{01\}^*$
 - Determine whether each of these strings is recognized by the deterministic finite-state automaton in Figure 1.
a) 111 b) 0011 c) 1010111 d) 011011011
 - Determine whether each of these strings is recognized by the deterministic finite-state automaton in Figure 1.
a) 010 b) 1101 c) 1111110 d) 010101010
 - Determine whether all the strings in each of these sets are recognized by the deterministic finite-state automaton in Figure 1.
a) $\{0\}^*$ b) $\{0\}\{0\}^*$ c) $\{1\}\{0\}^*$
d) $\{01\}^*$ e) $\{0\}^*\{1\}^*$ f) $\{1\}\{0, 1\}^*$
 - Show that if $M = (S, I, f, s_0, F)$ is a deterministic finite-state automaton and $f(s, x) = s$ for the state $s \in S$ and the input string $x \in I^*$, then $f(s, x^n) = s$ for every nonnegative integer n . (Here x^n is the concatenation of n copies of the string x , defined recursively in Exercise 37 in Section 5.3.)
 - Given a deterministic finite-state automaton $M = (S, I, f, s_0, F)$, use structural induction and the recursive definition of the extended transition function f to prove that $f(s, xy) = f(f(s, x), y)$ for all states $s \in S$ and all strings $x \in I^*$ and $y \in I^*$.
- In Exercises 16–22 find the language recognized by the given deterministic finite-state automaton.



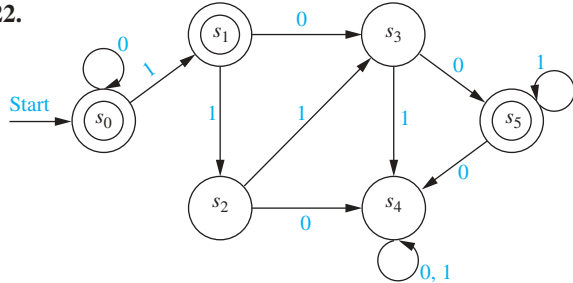
20.



21.



22.



23. Construct a deterministic finite-state automaton that recognizes the set of all bit strings beginning with 01.
24. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that end with 10.
25. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that contain the string 101.
26. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that do not contain three consecutive 0s.
27. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that contain exactly three 0s.
28. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that contain at least three 0s.
29. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that contain three consecutive 1s.
30. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that begin with 0 or with 11.
31. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that begin and end with 11.

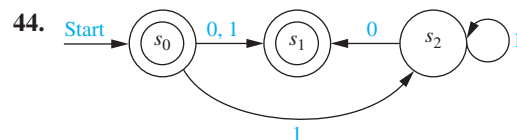
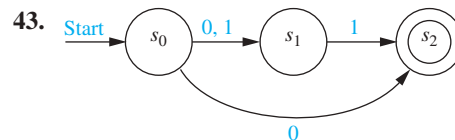
32. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that contain an even number of 1s.
33. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that contain an odd number of 0s.
34. Construct a deterministic finite-state automaton that recognizes the set of all bit strings that contain an even number of 0s and an odd number of 1s.
35. Construct a finite-state automaton that recognizes the set of bit strings consisting of a 0 followed by a string with an odd number of 1s.
36. Construct a finite-state automaton with four states that recognizes the set of bit strings containing an even number of 1s and an odd number of 0s.
37. Show that there is no finite-state automaton with two states that recognizes the set of all bit strings that have one or more 1 bits and end with a 0.
38. Show that there is no finite-state automaton with three states that recognizes the set of bit strings containing an even number of 1s and an even number of 0s.
39. Explain how you can change the deterministic finite-state automaton M so that the changed automaton recognizes the set $I^* - L(M)$.

40. Use Exercise 39 and finite-state automata constructed in Example 6 to find deterministic finite-state automata that recognize each of these sets.

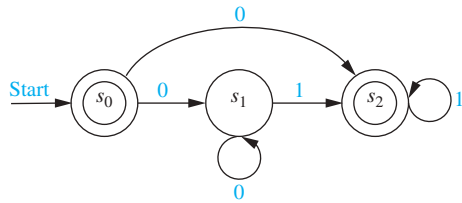
- a) the set of bit strings that do not begin with two 0s
- b) the set of bit strings that do not end with two 0s
- c) the set of bit strings that contain at most one 0 (that is, that do not contain at least two 0s)

41. Use the procedure you described in Exercise 39 and the finite-state automata you constructed in Exercise 25 to find a deterministic finite-state automaton that recognizes the set of all bit strings that do not contain the string 101.
42. Use the procedure you described in Exercise 39 and the finite-state automaton you constructed in Exercise 29 to find a deterministic finite-state automaton that recognizes the set of all bit strings that do not contain three consecutive 1s.

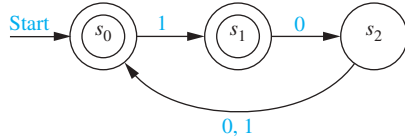
In Exercises 43–49 find the language recognized by the given nondeterministic finite-state automaton.



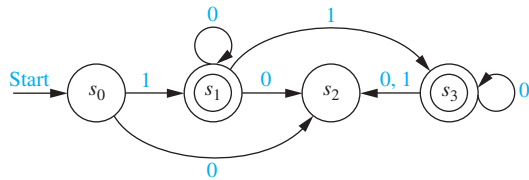
45.



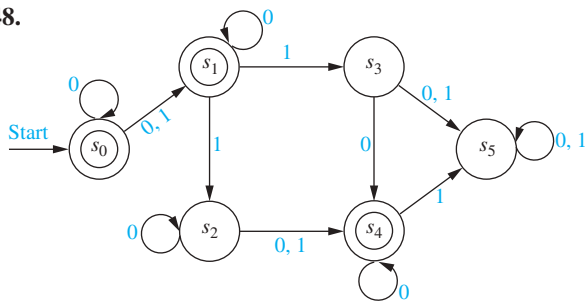
46.



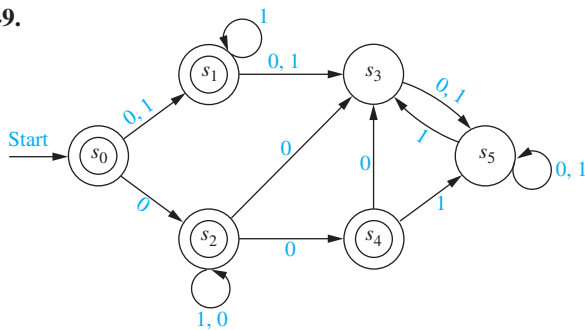
47.



48.



49.



50. Find a deterministic finite-state automaton that recognizes the same language as the nondeterministic finite-state automaton in Exercise 43.
51. Find a deterministic finite-state automaton that recognizes the same language as the nondeterministic finite-state automaton in Exercise 44.
52. Find a deterministic finite-state automaton that recognizes the same language as the nondeterministic finite-state automaton in Exercise 45.
53. Find a deterministic finite-state automaton that recognizes the same language as the nondeterministic finite-state automaton in Exercise 46.
54. Find a deterministic finite-state automaton that recognizes the same language as the nondeterministic finite-state automaton in Exercise 47.

55. Find a deterministic finite-state automaton that recognizes each of these sets.

- a) $\{0\}$ b) $\{1, 00\}$
 c) $\{1^n \mid n = 2, 3, 4, \dots\}$

56. Find a nondeterministic finite-state automaton that recognizes each of the languages in Exercise 55, and has fewer states, if possible, than the deterministic automaton you found in that exercise.

*57. Show that there is no finite-state automaton that recognizes the set of bit strings containing an equal number of 0s and 1s.

In Exercises 58–62 we introduce a technique for constructing a deterministic finite-state machine equivalent to a given deterministic finite-state machine with the least number of states possible. Suppose that $M = (S, I, f, s_0, F)$ is a finite-state automaton and that k is a nonnegative integer. Let R_k be the relation on the set S of states of M such that $sR_k t$ if and only if for every input string x with $l(x) \leq k$ [where $l(x)$ is the length of x , as usual], $f(s, x)$ and $f(t, x)$ are both final states or both not final states. Furthermore, let R_* be the relation on the set of states of M such that $sR_* t$ if and only if for every input string x , regardless of length, $f(s, x)$ and $f(t, x)$ are both final states or both not final states.

- *58. a) Show that for every nonnegative integer k , R_k is an equivalence relation on S . We say that two states s and t are **k -equivalent** if $sR_k t$.
 b) Show that R_* is an equivalence relation on S . We say that two states s and t are ***-equivalent** if $sR_* t$.
 c) Show that if s and t are two k -equivalent states of M , where k is a positive integer, then s and t are also $(k-1)$ -equivalent.
 d) Show that the equivalence classes of R_k are a refinement of the equivalence classes of R_{k-1} if k is a positive integer. (The refinement of a partition of a set is defined in the preamble to Exercise 49 in Section 9.5.)
 e) Show that if s and t are k -equivalent for every nonnegative integer k , then they are *-equivalent.
 f) Show that all states in a given R_* -equivalence class are final states or all are not final states.
 g) Show that if s and t are R_* -equivalent, then $f(s, a)$ and $f(t, a)$ are also R_* -equivalent for all $a \in I$.
- *59. Show that there is a nonnegative integer n such that the set of n -equivalence classes of states of M is the same as the set of $(n+1)$ -equivalence classes of states of M . Then show for this integer n , the set of n -equivalence classes of states of M equals the set of *-equivalence classes of states of M .

The **quotient automaton** $\bar{M}$ of the deterministic finite-state automaton $M = (S, I, f, s_0, F)$ is the finite-state automaton $(\bar{S}, I, \bar{f}, [s_0]_{R_*}, \bar{F})$, where the set of states $\bar{S}$ is the set of *-equivalence classes of S , the transition function $\bar{f}$ is defined by $\bar{f}([s]_{R_*}, a) = [f(s, a)]_{R_*}$ for all states $[s]_{R_*}$ of $\bar{M}$ and input symbols $a \in I$, and $\bar{F}$ is the set consisting of R_* -equivalence classes of final states of M .